Weekly Status Report

WEEK 5

UGP1 | MSE 496

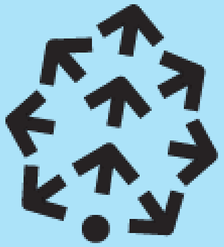
1

BACKEND

GENERAL STATUS

UPSERTING DOCUMENTS TO DB

VECTOR DB



METADATA

```
for i, (chunk, embedding) in enumerate(zip(chunks, embeddings)):
    vectors_to_upsert.append({
        "id": f"{paper_id}-chunk-{i}",
        "values": embedding.values,
        "metadata": {
            "paper_id": paper_id,
            "target_material": material,
            "process_type": process,
            "text": chunk
        }
    })
```

```
def chunk_text(text, chunk_size=300, overlap=50):
    """Word-based chunking strategy."""
    words = text.split()
```

CHUNK SIZE IS 300 AND OVERLAP IS 50

```
embeddings = pc.inference.embed(
    model="llama-text-embed-v2",
    inputs=chunks,
    parameters={"input_type": "passage", "truncate": "END"}
)
```

DEFAULT PINECONE EMBEDDING MODEL

```python
ass GeminiLLM:
  def generate_hyde_document(
      scope_paper_id: str | None,
  ) -> str:
      scope_line = scope_paper_id or "global corpus"
      return self._generate(
          system_instruction=(
              "Write a hypothetical but retrieval-optimized paragraph for an ALD literature "
              "search system. The paragraph should sound like a dense paper excerpt that would "
              "likely answer the user's question. Mention materials, precursors, process type, "
              "deposition windows, and characterization details only when relevant. Do not "
              "mention that the text is hypothetical. Do not use bullets."
          ),
          prompt=(
              f"Corpus scope: {scope_line}\n"
              f"Conversation:\n{_format_history(conversation)}\n\n"
              f"Question:\n{query}"
          ),
          max_output_tokens=250,
      )
```

generate_hyde_docume Aa ab .* 1 of 1

HYDE

```python
original_vector = self._embed(query, input_type="query")
original_chunks = self._query_index(
    vector=original_vector,
    top_k=self.settings.initial_top_k,
    scope_paper_id=request.scope_paper_id,
    source="original",
)
```

ORIGINAL CHUNKS

```python
if self.settings.hyde_enabled and self.llm_available:
    hyde_document = self._get_llm().generate_hyde_document(
        query=query,
        conversation=request.conversation,
        scope_paper_id=request.scope_paper_id,
    )
    if hyde_document:
        hyde_vector = self._embed(hyde_document, input_type="passage")
        hyde_chunks = self._query_index(
            vector=hyde_vector,
            top_k=self.settings.initial_top_k,
            scope_paper_id=request.scope_paper_id,
            source="hyde",
        )

fused_chunks = self._fuse_results(original_chunks, hyde_chunks)
reranked_chunks = self._rerank(query, fused_chunks)
```

FUSE HYDE + ORIGINAL CHUNKS

```python
def _rerank(self, query: str, candidates: list[RetrievedChunk]) -> list[RetrievedChunk]:
    if not candidates:
        return []

    pc = self._get_pinecone()
    documents = [
        {
            "id": chunk.source_id,
            "text": chunk.text,
            "paper_id": chunk.paper_id or "unknown",
            "target_material": chunk.target_material or "Unknown",
            "process_type": chunk.process_type or "Unknown",
        }
        for chunk in candidates
    ]

    reranked = pc.inference.rerank(
        model=self.settings.pinecone_rerank_model,
        query=query,
        documents=documents,
        top_n=min(self.settings.final_top_k, len(documents)),
        return_documents=True,
        parameters={"truncate": "END"},
    )

    index_lookup = {chunk.source_id: chunk for chunk in candidates}
    ordered: list[RetrievedChunk] = []

    for item in _safe_attr(reranked, "data", []) or []:
        document = _safe_attr(item, "document", {}) or {}
        source_id = document.get("id") or candidates[_safe_attr(item, "index", 0)].source_id
        chunk = index_lookup.get(source_id)
        if chunk is None:
            continue
        chunk.rerank_score = float(_safe_attr(item, "score", 0.0) or 0.0)
        ordered.append(chunk)

    return ordered
```

RE RANK THE FUSED CHUNKS

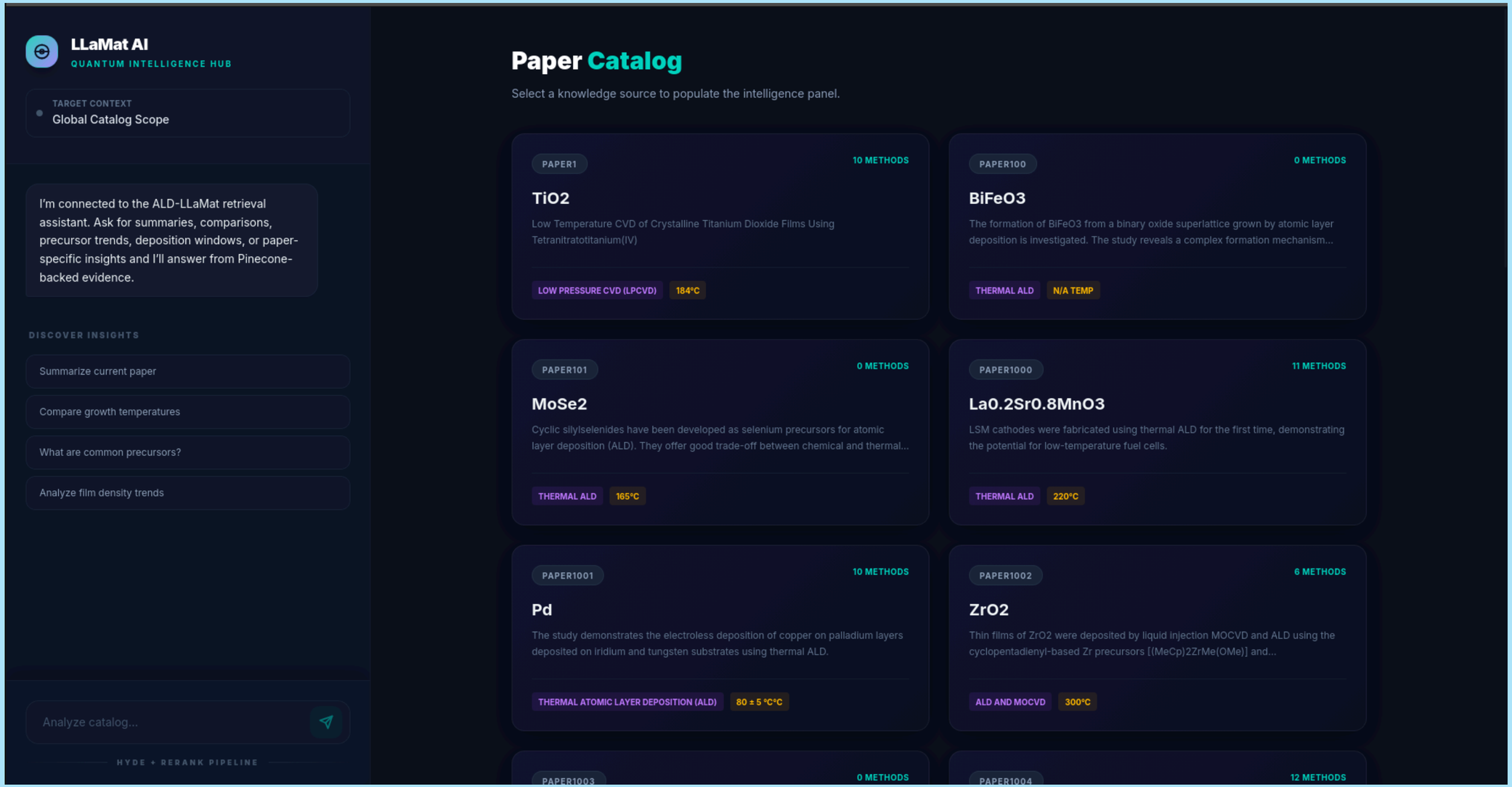
```python
if self.llm_available and reranked_chunks:
    context_blocks = self._build_context(reranked_chunks)
    answer = self._get_llm().generate_answer(
        query=query,
        conversation=request.conversation,
        scope_paper_id=request.scope_paper_id,
        context_blocks=context_blocks,
    )
else:
    answer = self._fallback_answer(query, reranked_chunks)
```

GENERATE OUTPUT USING LLM

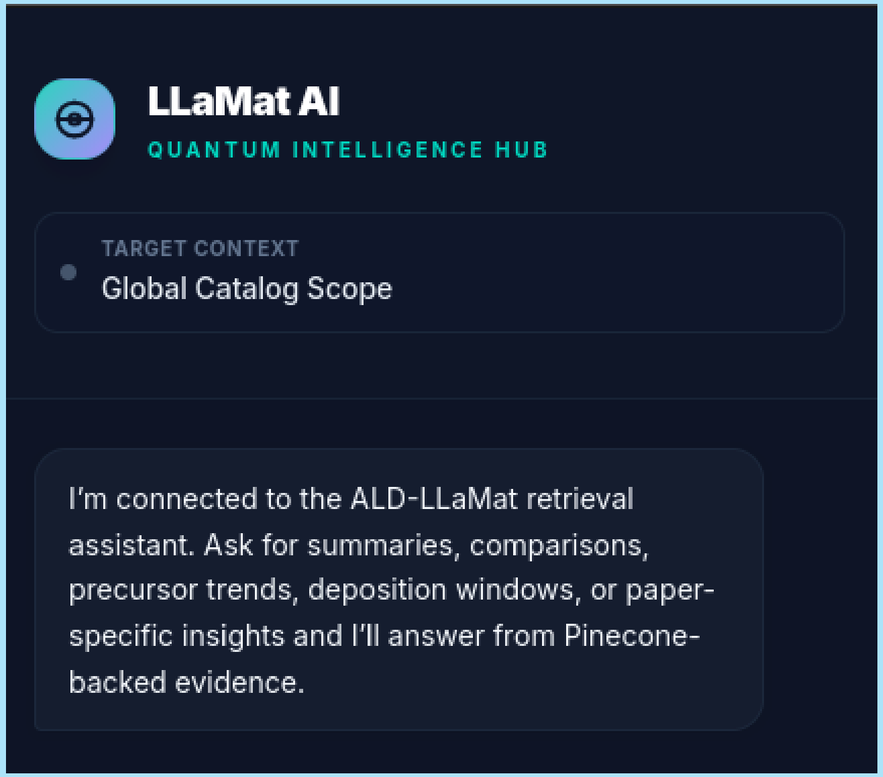
2

FRONT END AND DATABASES

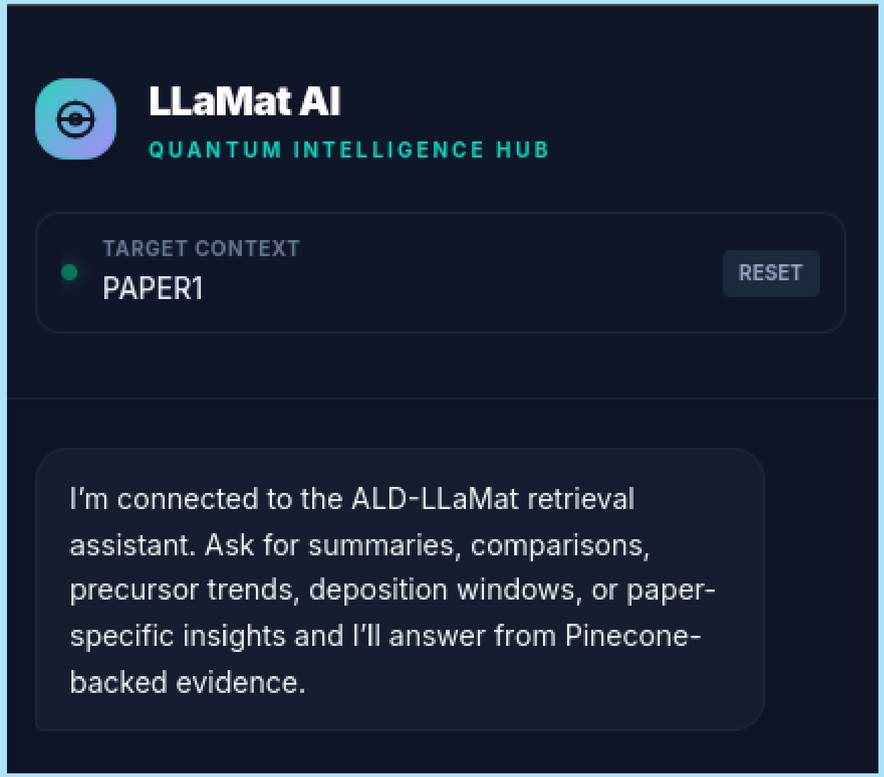
GENERAL STATUS



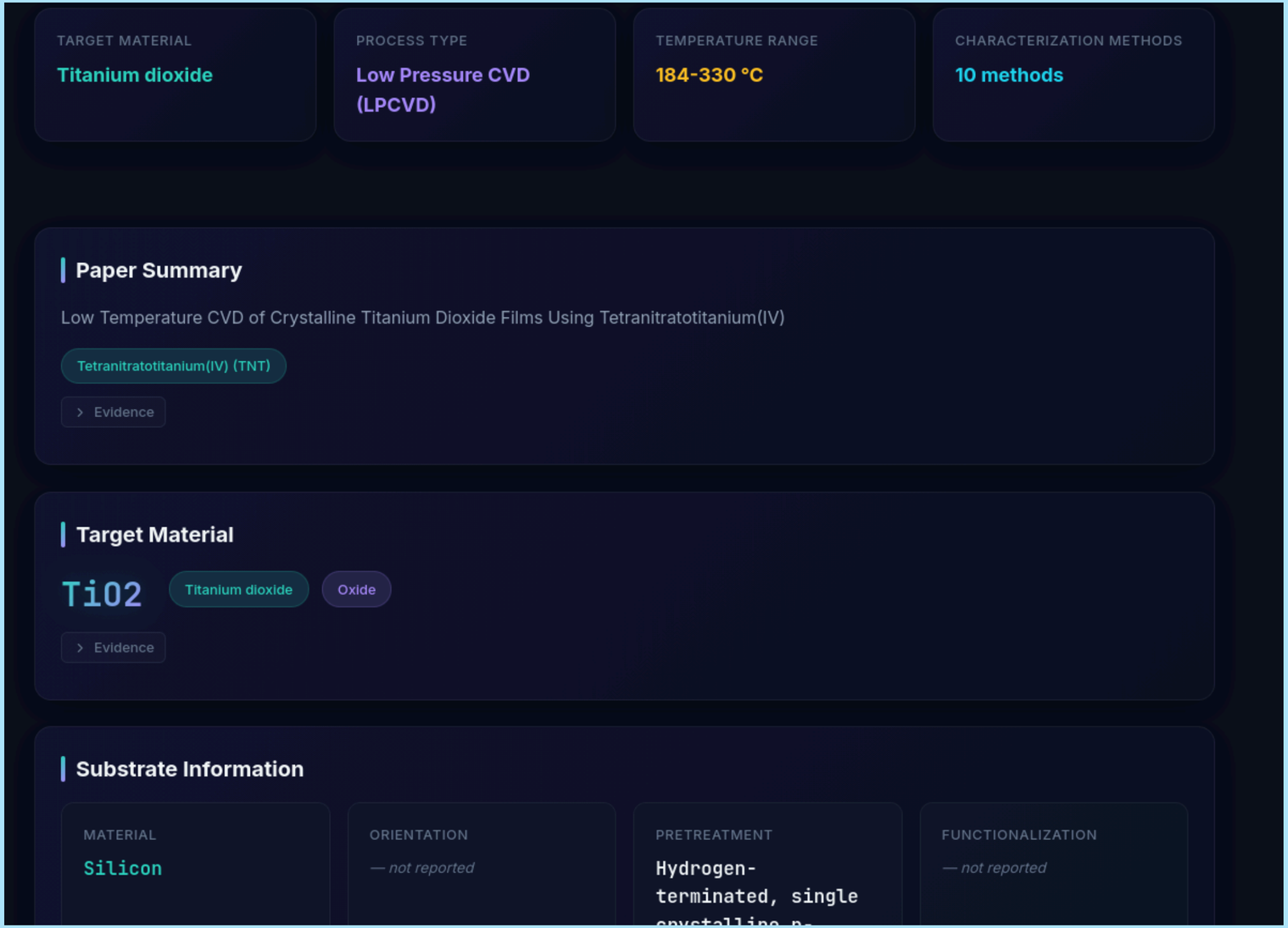
USER INTERFACE



GLOBAL CONTEXT



LOCAL CONTEXT



PAPER VISUALISATION TOOL

ALL INFO ABOUT THE PAPER

EXTRACTED USING LLAMA-3.2 3B

EXTRACTION TOOL FOR CONTENT.TXT FILES

```python
for folder in tqdm(folders[0:25]):
    if not folder.is_dir():
        continue
    if folder.name in completed_folders or folder.name in failed_folders:
        continue

    # FIX: check limit BEFORE processing, not after, to avoid overshooting
    if new_count >= max_new_folders:
        print(f"🔚 Reached limit of {max_new_folders} folders.")
        break

    try:
        success = process_paper(folder, output_dir)
        log_path = completed_log if success else failed_log
        with open(log_path, "a") as f:
            f.write(f"{folder.name}\n")
        if success:
            new_count += 1

    except Exception as e:
        print(f"⚠️  Failed on {folder.name}: {e}")
        traceback.print_exc()
        with open(failed_log, "a") as f:
            f.write(f"{folder.name}\n")
```

```python
def reaction_conditions_agent(fulltext: str, llm) -> Dict[str, Any]:  # FIX: correct
    prompt = _SYSTEM_PREAMBLE + f"""
Analyze the provided text to extract the Atomic Layer Deposition (ALD) surface reactic

Extraction Guidelines:
1. reaction_equations: Extract all formal chemical equations representing the surface
2. surface_mechanism_description: Provide a concise summary of the reaction steps as c
3. intermediate_species: Identify specific transient chemical species formed on the su
4. Evidence: Provide the exact excerpt from the text where these reactions or mechanis

Rules:
- If no formal equations are provided, return an empty list [] for "reaction_equations
- Maintain the original chemical stoichiometry as written in the text.
- Do not include conversational filler or markdown outside the JSON block.

Return the result STRICTLY as a valid JSON object:

{{
 "reaction_equations": [],
 "surface_mechanism_description": "",
 "intermediate_species": [],
 "evidence": ""
}}

"""  + "\nText:\n```" + fulltext + "```"
    output = llm.invoke(prompt)
    return robust_json_parse(output, default={
        "reaction_equations": [], "surface_mechanism_description": "",
        "intermediate_species": [], "evidence": None,
    })
```

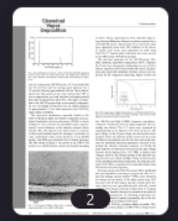
Back to Catalog

Hide PDF

TARGET MATERIAL	PROCESS TYPE	TEMPERATURE RANGE	CHARACTERIZATION METHODS
Titanium dioxide	Low Pressure CVD (LPCVD)	184-330 °C	10 methods

1 of 3 Automatic Zoom

Pages



Chemical Vapor Deposition

Communications

Low Temperature CVD of Crystalline Titanium Dioxide Films Using Tetranitratotitanium(IV)**

By *David C. Gilmer, Daniel G. Colombo, Charles J. Taylor, Jeff Roberts, Greg Haugstad, Stephen A. Campbell, Hyeon-Seag Kim, Glen D. Wilk, Michael A. Gribelyuk*, and *Wayne L. Gladfelter**

The continuing push to decrease the size of microelectronic devices is hampered by some of the physical properties of the current materials. Silicon dioxide is currently used as the gate dielectric in metal oxide semiconductor field effect transistors (MOSFETs), and operation of this device requires that the thickness of the dielectric be scaled along with the length of the gate between the source and drain. As the gate lengths approach 0.1 µm, the required SiO_2 thickness will drop to 15–20 Å, and leakage current through the dielectric will rise to unacceptable limits.[1] One alternative is to replace SiO_2 (dielectric constant k = 3.9) with a material having a higher dielectric constant that will allow the use of thicker, less leaky, films. Towards this end compounds such as Ta_2O_5 ($20 \leq k \leq 25$) have been evaluated.[2] We have recently reported on the first successful

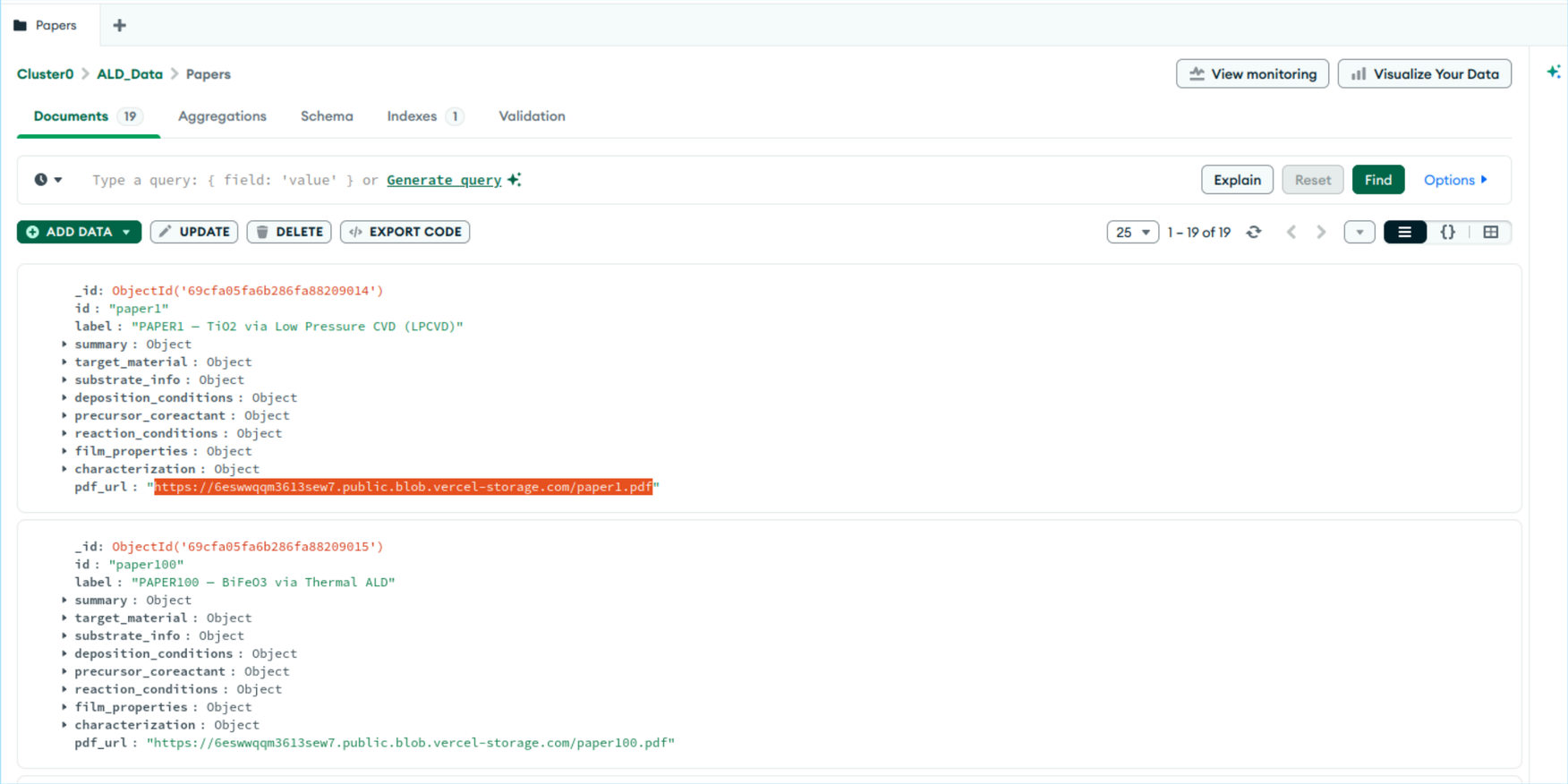
will deposit pure, crystalline TiO_2 at low temperature without requiring a separate source of oxygen.

Anhydrous metal nitrates vary widely in their properties, however, some have been shown to be volatile and to form metal oxides upon thermolysis.[34,35] The structure of tetranitratotitanium(IV), TNT, which was synthesized using a literature procedure,[36] consists of four bidentate nitrato ligands arranged in a slightly flattened tetrahedral pattern around the central titanium atom (see Fig. 1).[37] It is a white, crystalline solid, purified by distillation, that has a melting point of 58 °C and that readily sublimes at 50 °C and 0.2 torr.

Fig. 1. The structure of $Ti(NO_3)_4$, TNT.

Low pressure CVD (LPCVD) using TNT was accom-

CAN VIEW THE ORIGINAL PAPER AS WELL



DB FOR PAPER VISUALISATION TOOL

```
const pdfPath = path.join(
  process.cwd(),
  "..",
  "Web Scrapper",
  "ald_papers_naming",
  `${id}.pdf`
);

if (!fs.existsSync(pdfPath)) {
  return new NextResponse(`PDF not found: ${id}.pdf`, { status: 404 });
}

const fileBuffer = fs.readFileSync(pdfPath);

return new NextResponse(fileBuffer, {
  headers: {
    "Content-Type": "application/pdf",
    "Content-Disposition": `inline; filename="${id}.pdf"`,
    "Content-Length": fileBuffer.length.toString(),
  },
});
```

VERCEL BLOB FOR STORING THE PDFS

Thank you